OBSERVACIONES DE LA PRÁCTICA

Juan Sebastián Rojas Marín, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan José Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

a) ¿Qué estructura de datos subyacente se utiliza comúnmente para implementar una cola de prioridad en Python?

Normalmente se usa un heap binario. Es un árbol binario completo en el que cada padre tiene mejor prioridad que sus hijos, así que el elemento más importante siempre queda arriba, en la raíz. Como el árbol se va llenando por niveles y sin dejar huecos, se puede guardar en una lista sin usar apuntadores. Por ejemplo, el módulo heapq de Python usa una lista que empieza en la posición 0, donde los hijos de la posición k están en 2k + 1 y 2k + 2. En nuestro laboratorio lo implementamos sobre una array_list y dejamos la posición 0 vacía, entonces el padre de k queda en k // 2 y sus hijos en 2k y 2k + 1.

b) ¿Cuál es la diferencia entre una cola FIFO tradicional y una cola de prioridad?

En una cola FIFO los elementos se atienden en el mismo orden en que llegan: el primero que entra es el primero que sale. En una cola de prioridad, en cambio, se atiende primero al que tenga mejor prioridad, sin importar cuándo llegó. En nuestro laboratorio lo vimos claramente: la ruta 10, que es la primera del archivo, sale a las 05:00, mientras que la ruta 112A aparece mucho más abajo pero sale a las 00:15. Aun así, la cola atendió primero a la 112A, porque ordena por la hora del primer bus y no por el orden en que se cargaron los datos.

c) ¿Qué módulo proporciona Python para trabajar fácilmente con colas de prioridad?

Python tiene el módulo heapq, que trabaja sobre una lista normal y la maneja como un heap de mínimo, o sea que el elemento más pequeño siempre queda de primero. Sus funciones principales son heappush, para insertar un elemento sin romper el orden del heap; heappop, para sacar el menor; y heapify, para convertir una lista que ya existe en un heap. Además, está la clase PriorityQueue del módulo queue, que usa heapq por dentro y está pensada para programas en los que varios hilos comparten la misma cola.

d) ¿Qué ventajas tiene el uso de una cola de prioridad sobre una lista ordenada manualmente?

Si usamos una lista ordenada, cada vez que insertamos algo tenemos que buscar dónde va y correr los demás elementos, y eso cuesta O(n). Con una cola de prioridad basada en heap, insertar y sacar cuestan O(log n), que es mucho más rápido cuando hay muchos datos. Además, el heap no necesita tener todo ordenado: solo se asegura de que el de mejor prioridad esté de primero, que es justo lo que necesitamos para atender en orden.

e) Si dos elementos tienen la misma prioridad, ¿cómo decide la cola cuál atender primero?

El heap no garantiza un orden fijo entre elementos que tienen la misma prioridad. El que sale primero depende de cómo se fueron acomodando al insertar y eliminar. En nuestros datos hay cinco rutas que salen a las 05:00 (la 10, la 119, la 154, la 232 y la 293), y aunque la ruta 10 es la primera del archivo, la cola atendió antes a la 119 y a la 232. Esto está bien, porque el enunciado permite que las rutas con la misma prioridad salgan en cualquier orden. Si necesitáramos un orden fijo, tendríamos que agregar un criterio de desempate, por ejemplo el orden de llegada.

f) ¿Qué situaciones del mundo real se pueden modelar con colas de prioridad? Mencione al menos dos.

Un ejemplo es la atención en urgencias de un hospital, donde se atiende primero al paciente más grave y no al que llegó primero. Otro es la forma en que un sistema operativo decide qué proceso ejecutar, dándole el turno primero a los de mayor prioridad. También se usan en aplicaciones de mapas para encontrar la ruta más corta y, como en este laboratorio, para organizar rutas de buses según la hora en que salen.

g) En un sistema de atención médica, ¿cómo se puede usar una cola de prioridad para organizar a los pacientes?

A cada paciente se le puede asignar una prioridad según qué tan grave esté, por ejemplo de 1 (crítico) a 5 (leve). Usaríamos una cola de prioridad orientada a menor, para que siempre se atienda primero al que tenga el número más bajo, es decir, al más urgente. Y si un paciente empeora mientras espera, le mejoramos la prioridad y sube en la cola, sin tener que reorganizar a todos los demás.

h) ¿Cómo afectaría al rendimiento usar una lista simple en lugar de una estructura especializada como heap_pq para manejar prioridades?

Si usamos una lista sin ordenar, insertar es rápido, pero para encontrar el de mayor prioridad toca recorrerla toda, lo que cuesta O(n). Si la lista está ordenada pasa lo contrario: sacar es rápido, pero cada inserción cuesta O(n). Con un heap, las dos operaciones cuestan O(log n). Con pocos datos casi no se nota la diferencia, pero con muchos sí: si hacemos muchas inserciones y eliminaciones, con una lista el costo total puede llegar a $O(n^2)$, mientras que con el heap se queda en O(n log n).

i) ¿Qué complejidad temporal tiene la inserción y eliminación/atención en una cola de prioridad basada en heap?

Las dos cuestan O(log n). Cuando insertamos, el elemento entra al final y va subiendo hasta su lugar; cuando eliminamos, el último pasa a la raíz y va bajando hasta donde le corresponde. En los dos casos, lo máximo que recorre es la altura del árbol, que es log n porque el heap es un árbol binario completo. Y consultar el de mayor prioridad cuesta O(1), porque siempre está en la primera posición.